

Connect Streamlit to Google BigQuery

Introduction

This guide explains how to securely access a BigQuery database from Streamlit Community Cloud. It uses the [google-cloud-bigquery](#) library and Streamlit's [Secrets management](#).

Create a BigQuery database

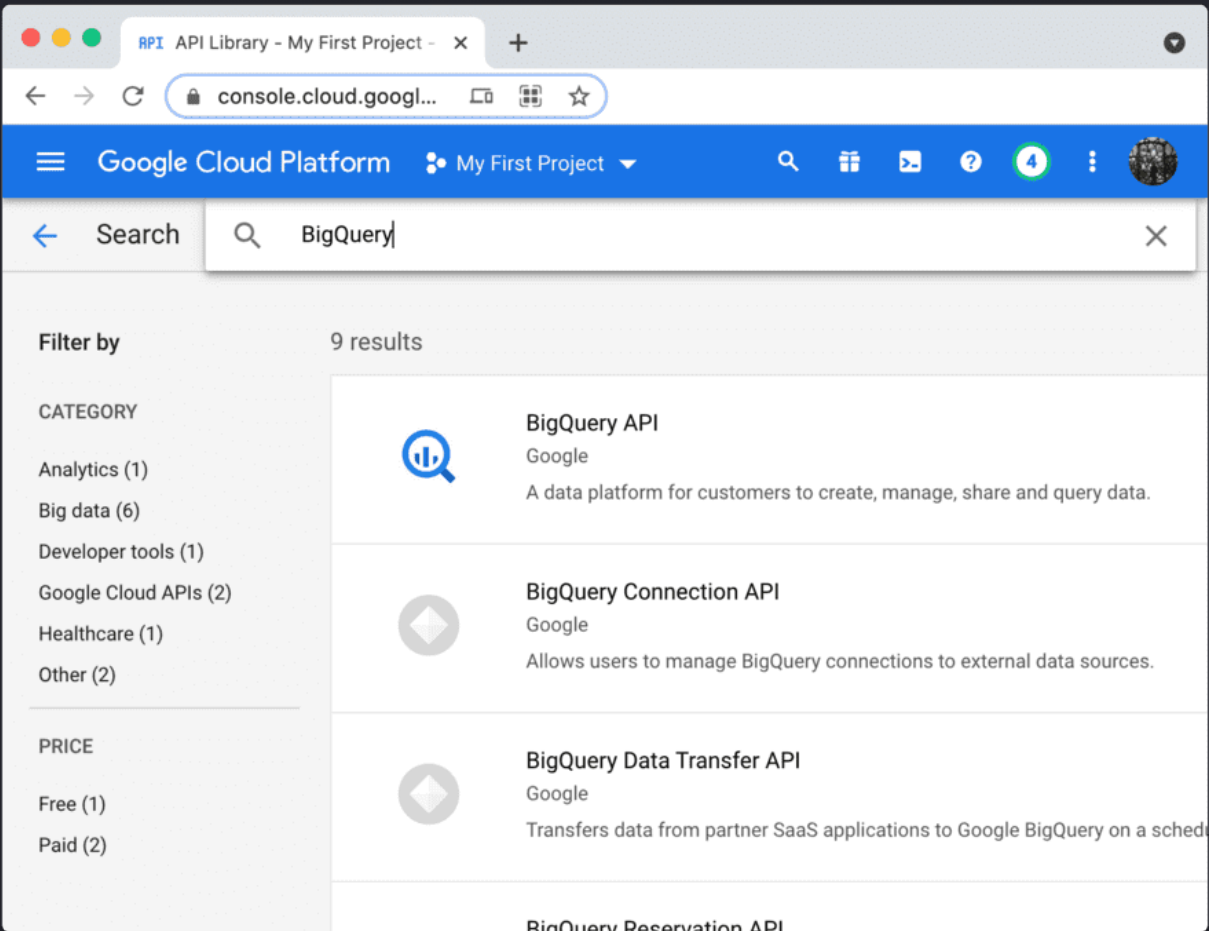
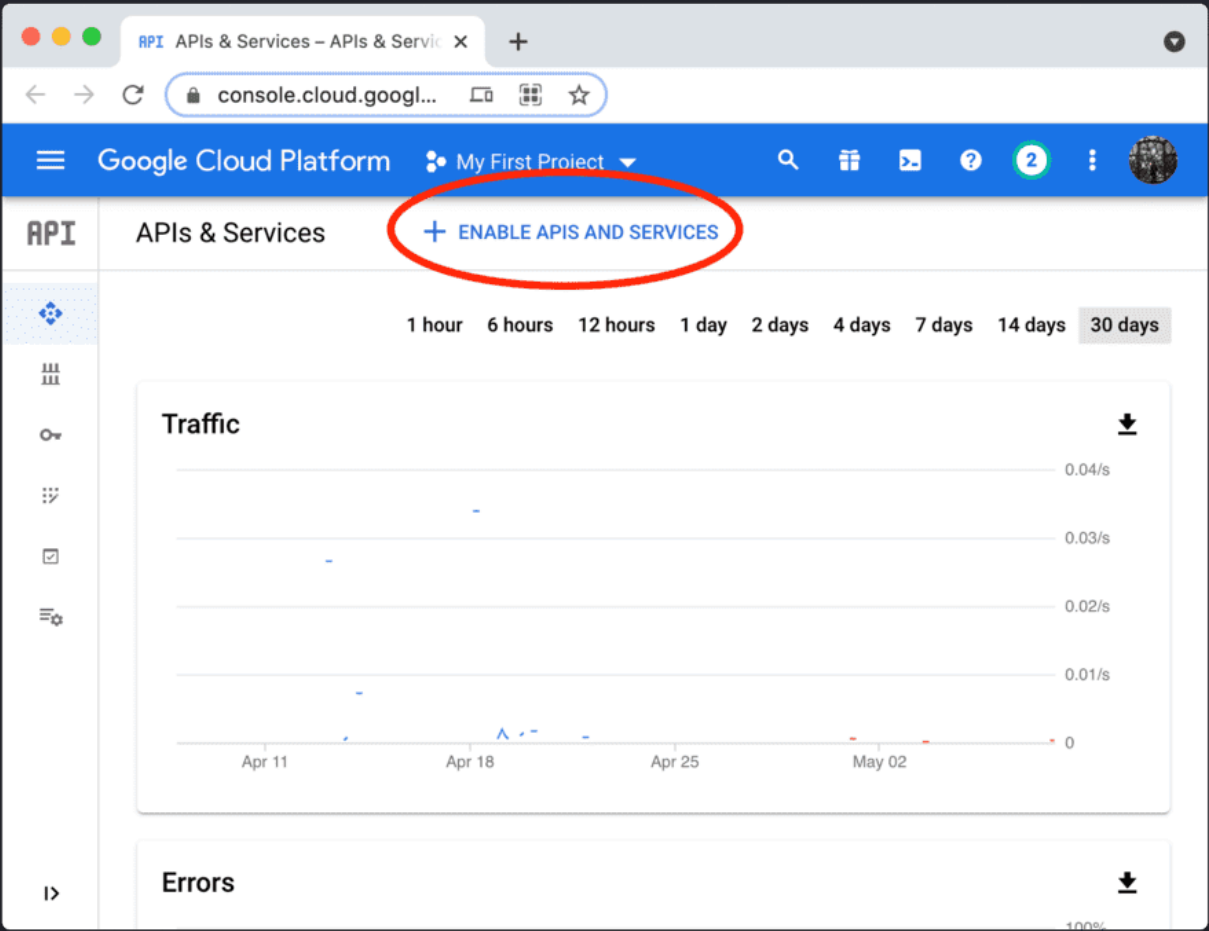


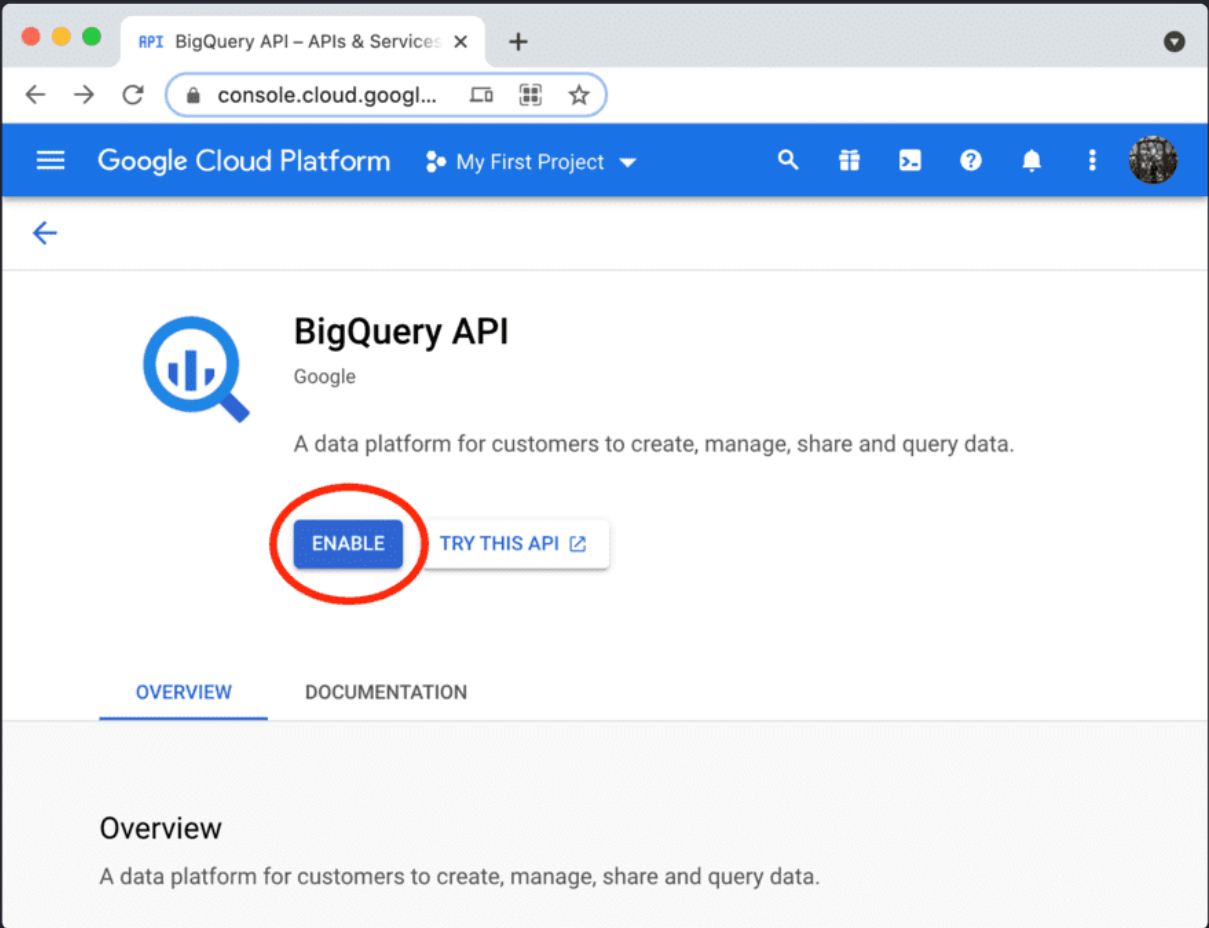
If you already have a database that you want to use, feel free to [skip to the next step](#).

For this example, we will use one of the [sample datasets](#) from BigQuery (namely the `shakespeare` table). If you want to create a new dataset instead, follow [Google's quickstart guide](#).

Enable the BigQuery API

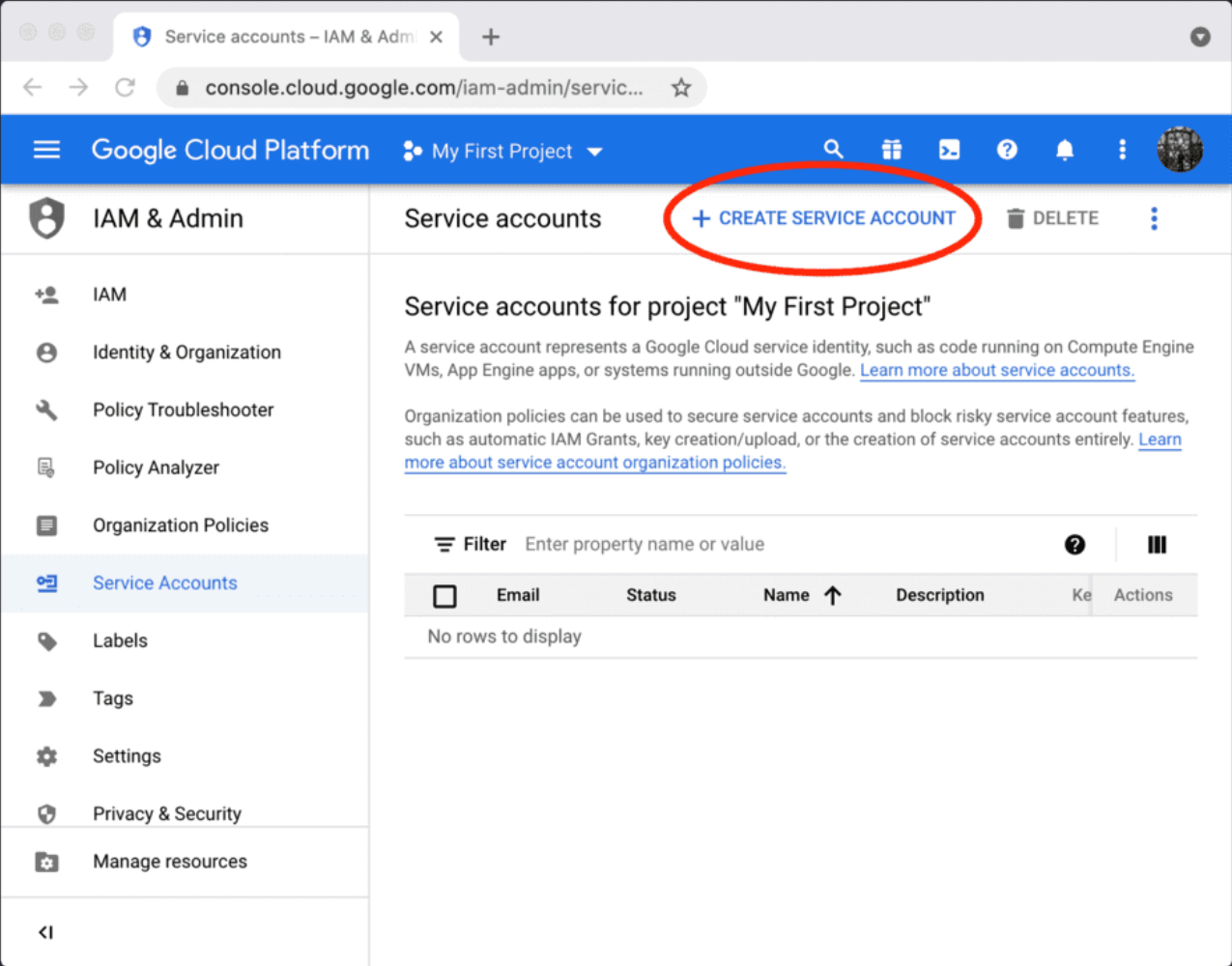
Programmatic access to BigQuery is controlled through [Google Cloud Platform](#). Create an account or sign in and head over to the [APIs & Services dashboard](#) (select or create a project if asked). As shown below, search for the BigQuery API and enable it:

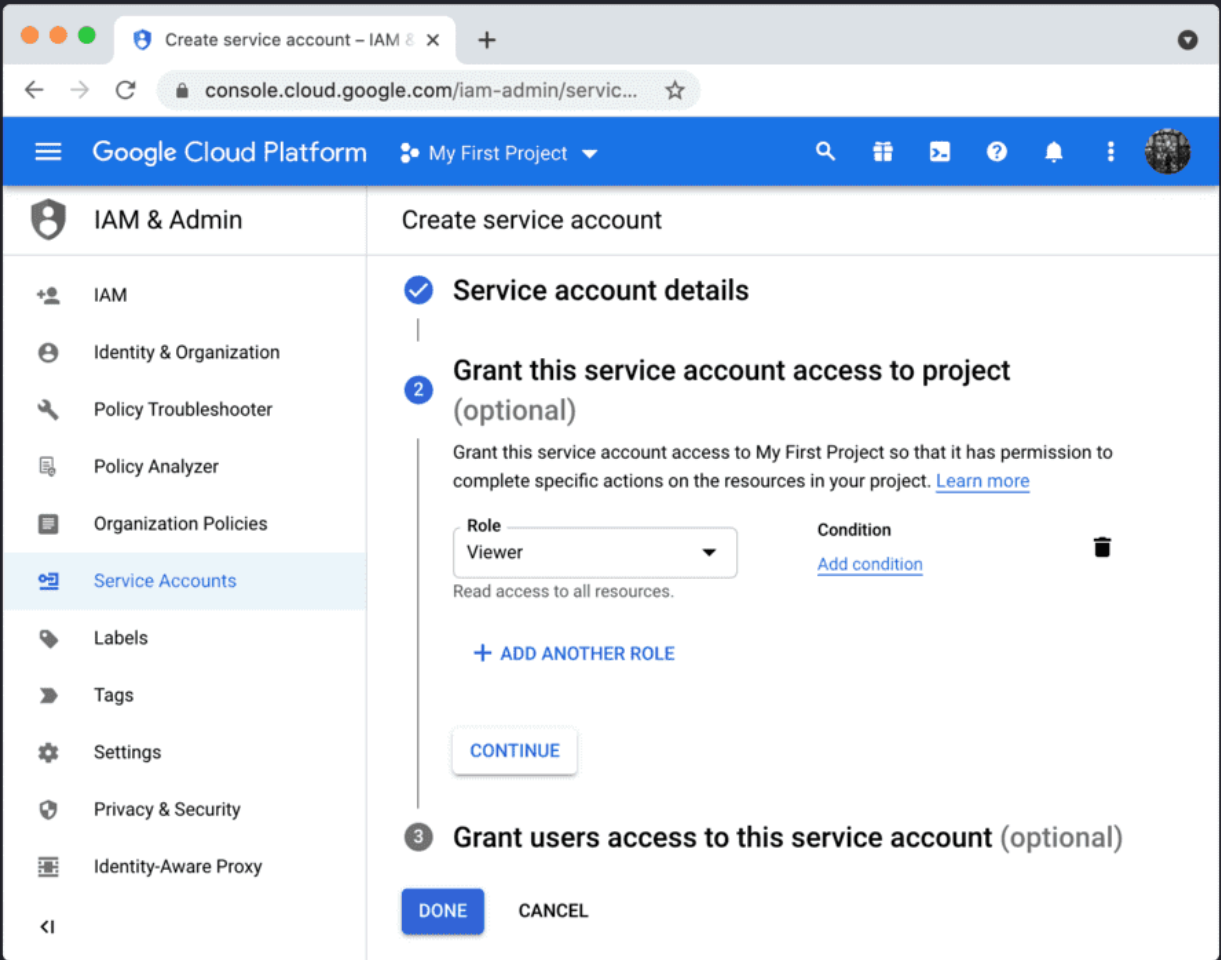
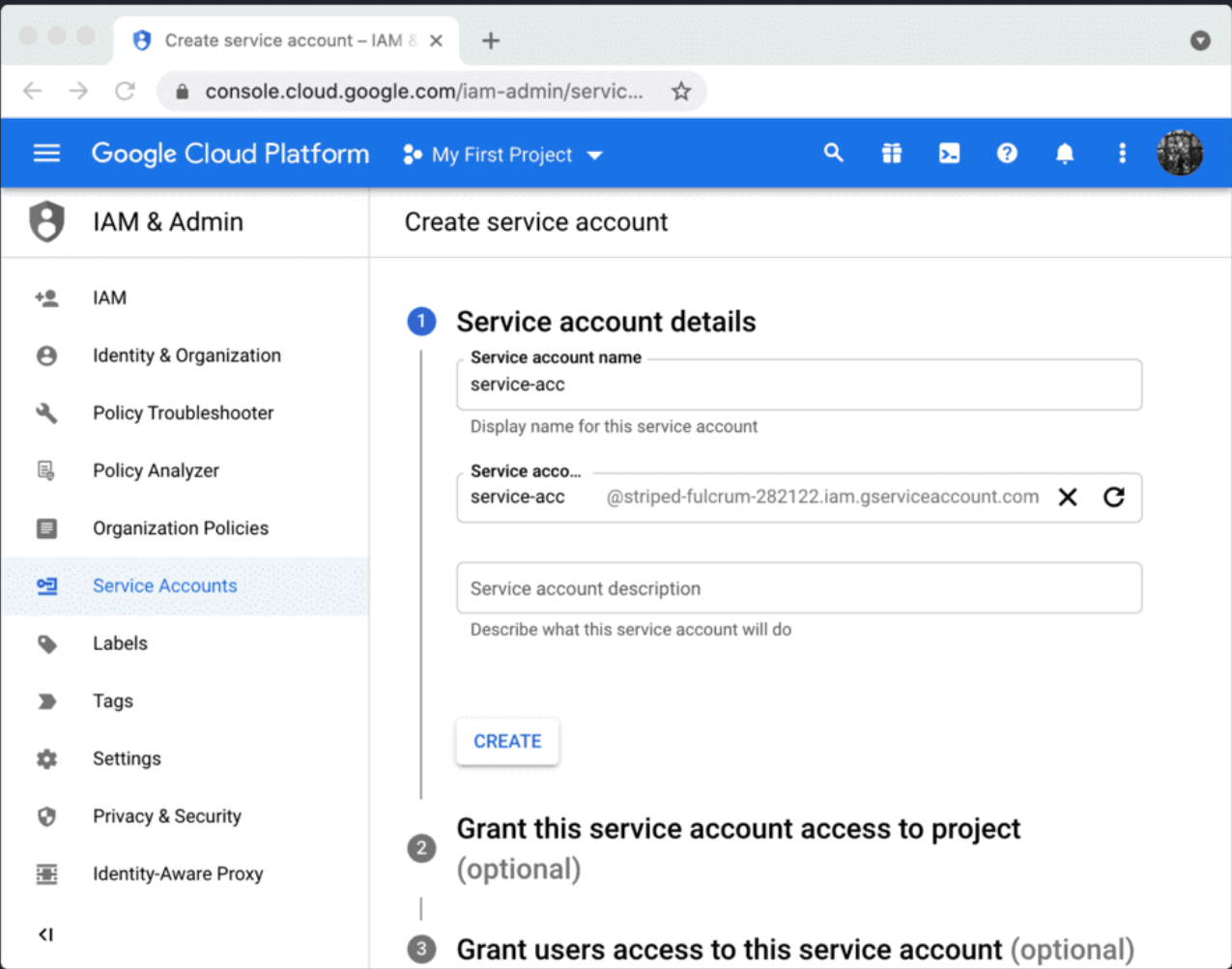




Create a service account & key file

To use the BigQuery API from Streamlit Community Cloud, you need a Google Cloud Platform service account (a special account type for programmatic data access). Go to the [Service Accounts](#) page and create an account with the **Viewer** permission (this will let the account access data but not change it):

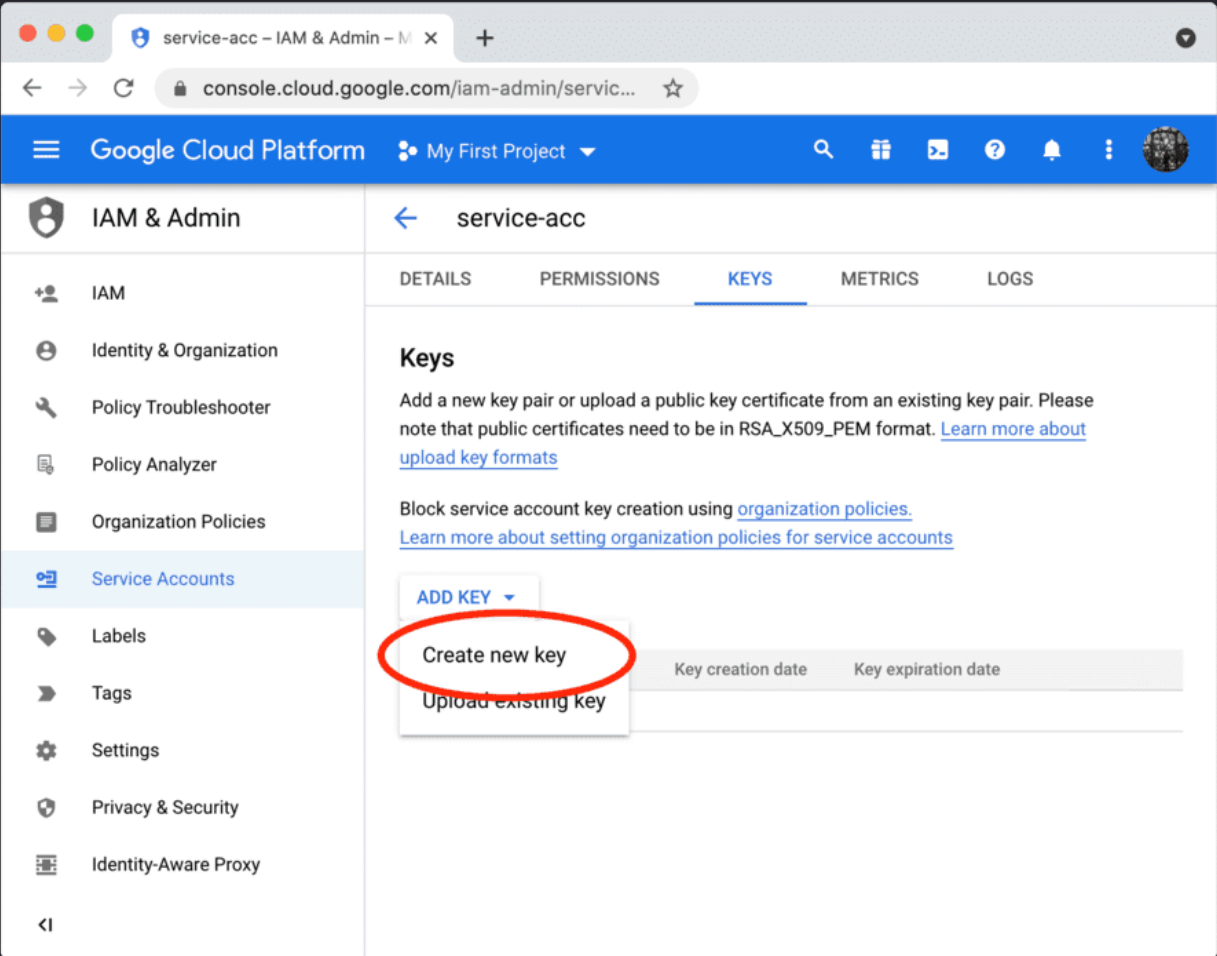
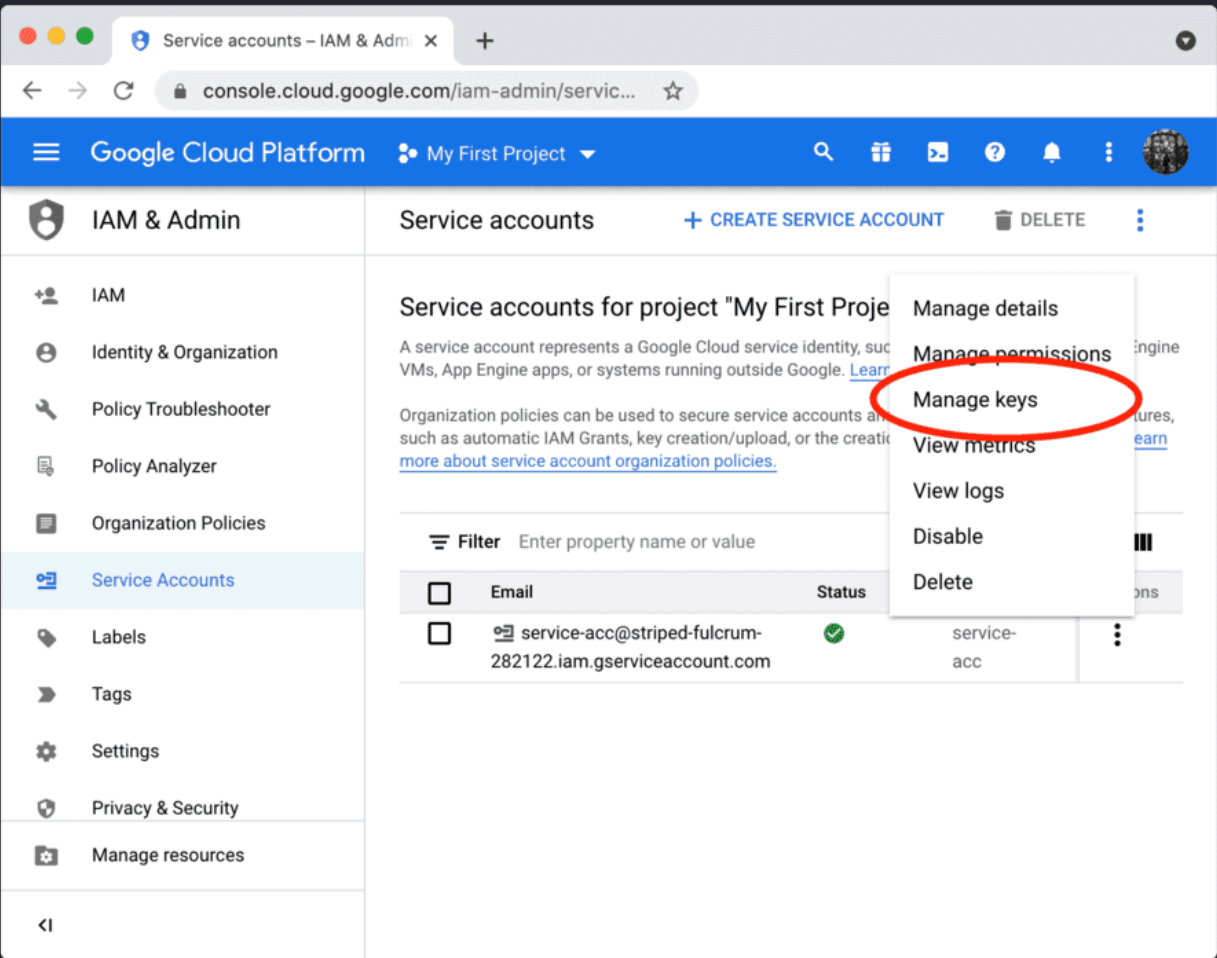


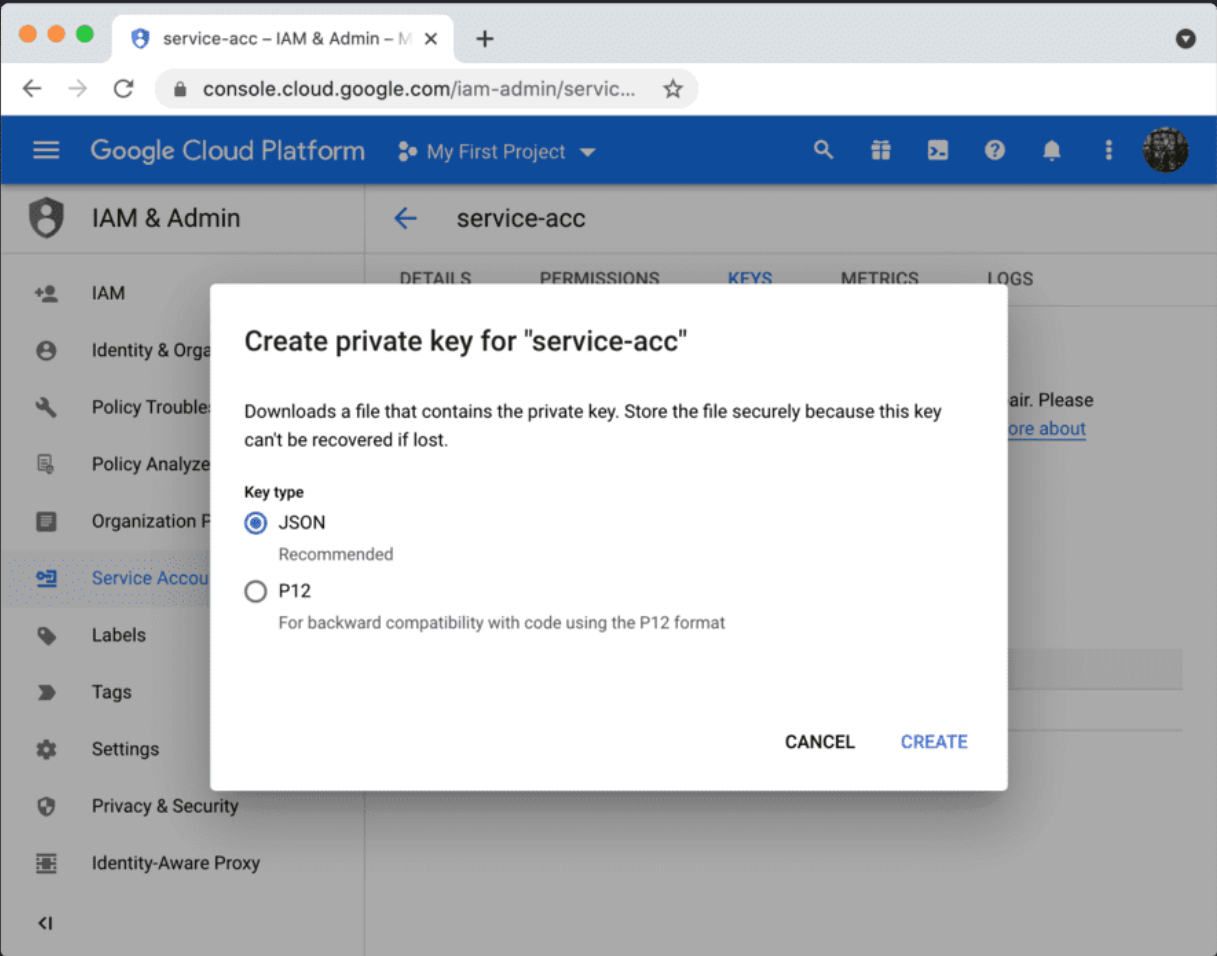


 Note

If the button **CREATE SERVICE ACCOUNT** is gray, you don't have the correct permissions. Ask the admin of your Google Cloud project for help.

After clicking **DONE**, you should be back on the service accounts overview. Create a JSON key file for the new account and download it:





Add the key file to your local app secrets

Your local Streamlit app will read secrets from a file `.streamlit/secrets.toml` in your app's root directory. Create this file if it doesn't exist yet and add the content of the key file you just downloaded to it as shown below:

```
# .streamlit/secrets.toml

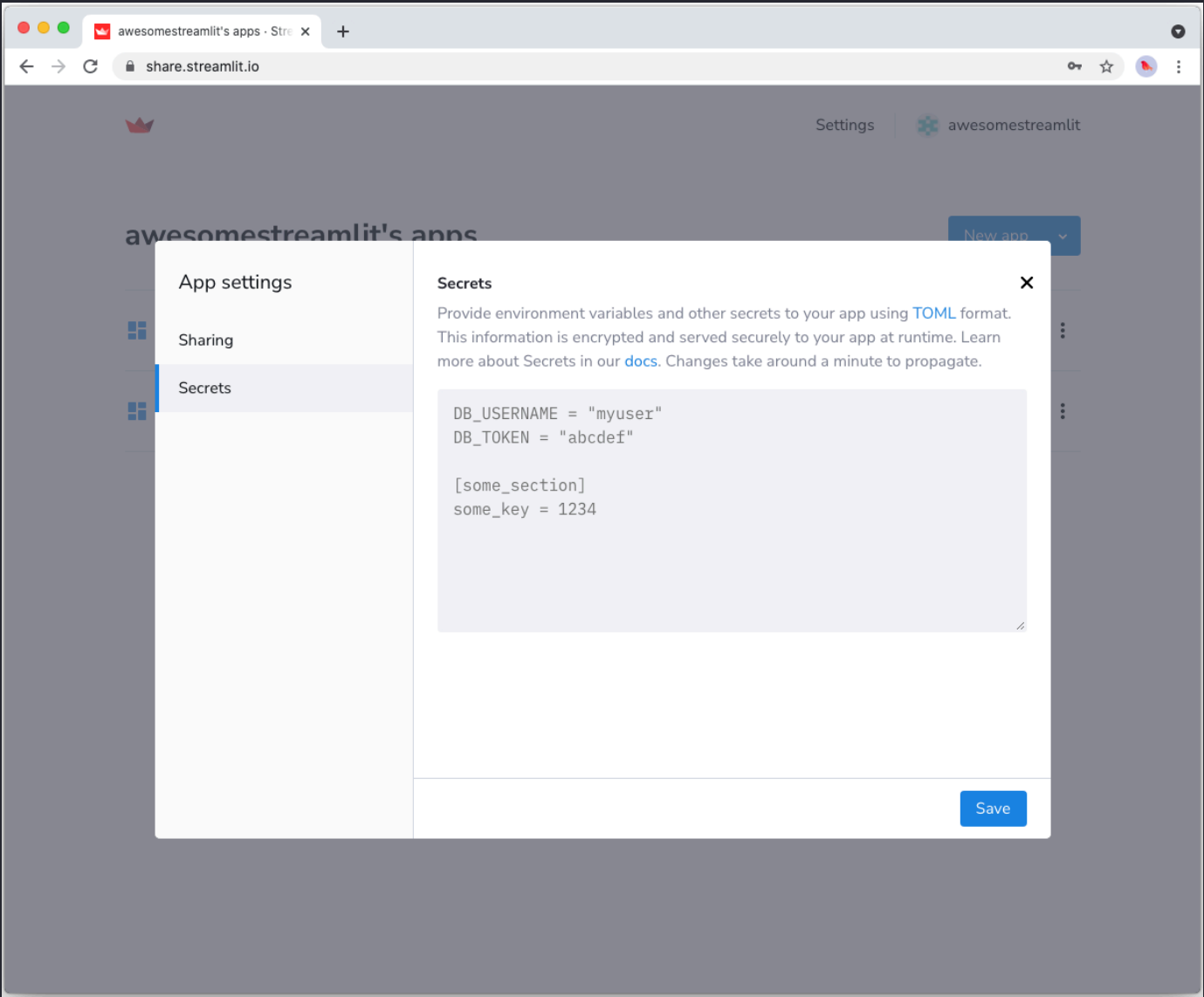
[gcp_service_account]
type = "service_account"
project_id = "xxx"
private_key_id = "xxx"
private_key = "xxx"
client_email = "xxx"
client_id = "xxx"
auth_uri = "https://accounts.google.com/o/oauth2/auth"
token_uri = "https://oauth2.googleapis.com/token"
auth_provider_x509_cert_url = "https://www.googleapis.com/oauth2/v1/certs"
client_x509_cert_url = "xxx"
```

 Important

Add this file to `.gitignore` and don't commit it to your GitHub repo!

Copy your app secrets to the cloud

As the `secrets.toml` file above is not committed to GitHub, you need to pass its content to your deployed app (on Streamlit Community Cloud) separately. Go to the [app dashboard](#) and in the app's dropdown menu, click on **Edit Secrets**. Copy the content of `secrets.toml` into the text area. More information is available at [Secrets management](#).



Add google-cloud-bigquery to your requirements file

Add the [google-cloud-bigquery](#) package to your `requirements.txt` file, preferably pinning its version (replace `x.x.x` with the version want installed):

```
# requirements.txt
google-cloud-bigquery==x.x.x
```

Write your Streamlit app

Copy the code below to your Streamlit app and run it. Make sure to adapt the query if you don't use the sample table.

```
# streamlit_app.py

import streamlit as st
from google.oauth2 import service_account
from google.cloud import bigquery
```

```

# Create API client.
credentials = service_account.Credentials.from_service_account_info(
    st.secrets["gcp_service_account"]
)
client = bigquery.Client(credentials=credentials)

# Perform query.
# Uses st.cache_data to only rerun when the query changes or after 10 min.
@st.cache_data(ttl=600)
def run_query(query):
    query_job = client.query(query)
    rows_raw = query_job.result()
    # Convert to list of dicts. Required for st.cache_data to hash the return value.
    rows = [dict(row) for row in rows_raw]
    return rows

rows = run_query("SELECT word FROM `bigquery-public-data.samples.shakespeare`")

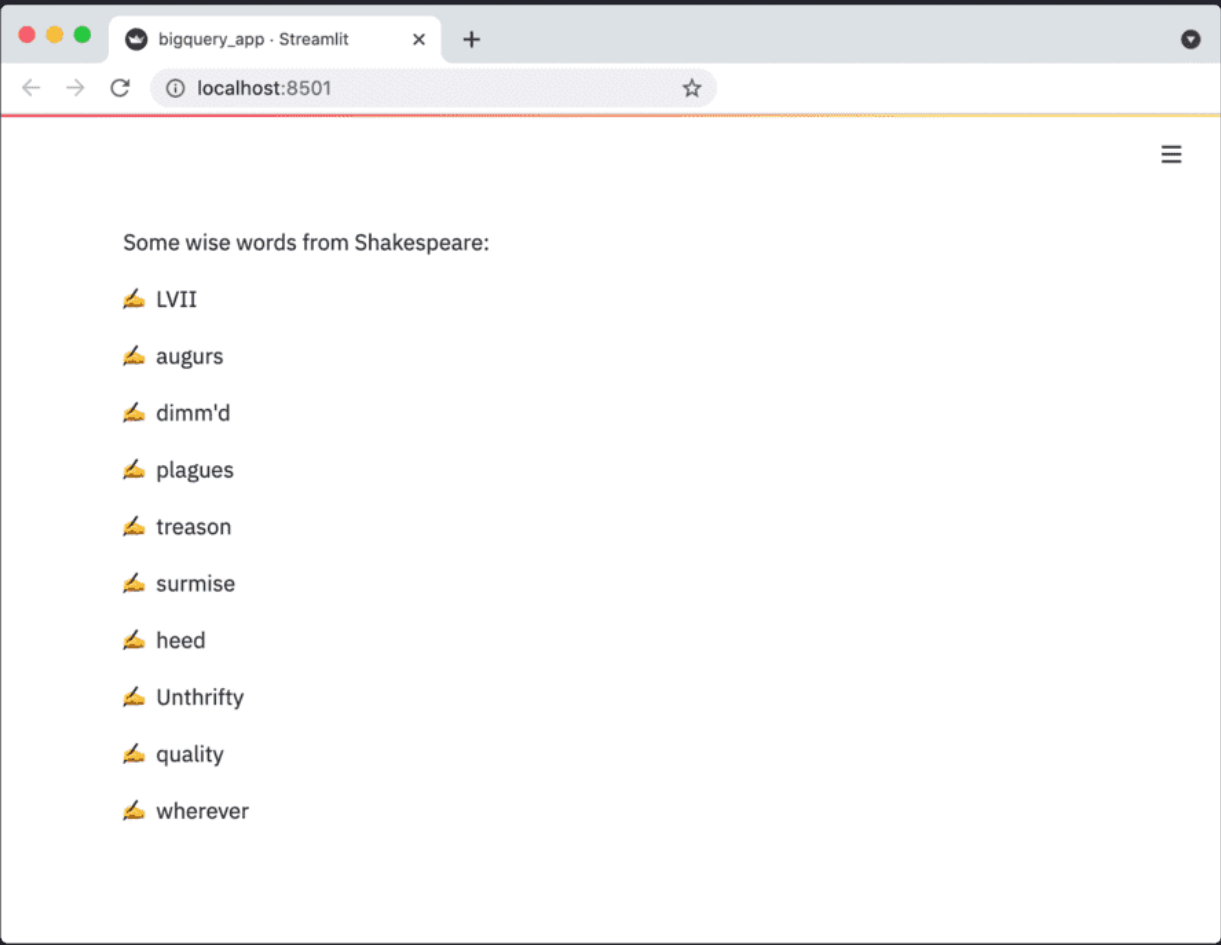
# Print results.
st.write("Some wise words from Shakespeare:")
for row in rows:
    st.write("👉 " + row['word'])

```

See `st.cache_data` above? Without it, Streamlit would run the query every time the app reruns (e.g. on a widget interaction). With `st.cache_data`, it only runs when the query changes or after 10 minutes (that's what `ttl` is for). Watch out: If your database updates more frequently, you should adapt `ttl` or remove caching so viewers always see the latest data. Learn more in [Caching](#).

Alternatively, you can use pandas to read from BigQuery right into a dataframe! Follow all the above steps, install the [pandas-gbq](#) library (don't forget to add it to `requirements.txt`!), and call `pandas.read_gbq(query, credentials=credentials)`. More info in [the pandas docs](#).

If everything worked out (and you used the sample table), your app should look like this:



← Previous: AWS S3

Next: Firestore →



Still have questions?

Our [forums](#) are full of helpful information and Streamlit experts.